

DON'T GENERATE THE GRAPH. DECIDE IT.

Extract a knowledge graph with a Jev-style decision model on Ollama

Most tools ask an AI to write a knowledge graph and hope it is right. This one asks small yes-or-no questions, one link at a time, and keeps only the answers the model is sure about. The rest wait for a person.

Raphael MANSUY · linkedin.com/in/raphaelmansuy · 3 October 2026

A CHAT MODEL WRITES THE ANSWER

~250 tokens

written for each sentence (a token is about $\frac{3}{4}$ of a word)

- It writes every link it can think of
- It does not say how sure it is
- A guess looks the same as a fact

A DECISION MODEL ANSWERS A QUESTION

3 tokens

for two complete answers

Does EdgeQuake use PostgreSQL? **0.99**

Did Acme Inc found PostgreSQL? **0.01**

A real call to `tev1` on one computer, 2.9 seconds. 0.99 means "very sure: yes".

2.3×

**fewer wrong links than
Mistral Small**

On a public news test of 288 sentences it kept 218 wrong links against 504. It found 154 right links to Mistral's 183.

10×

**faster than a chat model on
the same computer**

One seven-sentence note: 7.9 seconds for `tev1` against 77.8 for `gemma4`.

0

**invented kinds of name or
link**

Everything must come from the list you wrote. A link your list forbids is never even asked.

What it does not do. A model trained on that same test still scores higher (0.89 on names and 0.71 on links, against 0.69 and 0.39 here). A chat model on a hosted service can be faster: 3.5 seconds on that note. And a high number can still be wrong: it kept both "Ollama uses Nimble" (right, 0.97) and the reversed "Nimble uses Ollama" (wrong, 0.91). This article shows all of it.

CONTENTS

- 01 The problem, in one minute
- 02 Why it is hard to do well today
- 03 The idea: ask small questions, get a number
- 04 How edgextract works
- 05 Write your own list in ten minutes
- 06 What we measured
- 07 Where it gets things wrong
- 08 See the graph
- 09 Which one should you use
- 10 Try it

01

The problem, in one minute

Your company already knows a lot. It is written down in notes, tickets, wikis, and release posts. A person can read one page and answer “Who works where?” A computer cannot, because the facts are buried inside sentences.

Jane Doe joined Acme Inc in Berlin. Acme Inc uses EdgeQuake. EdgeQuake depends on PostgreSQL.

A **knowledge graph** fixes that. It is a list of things (people, companies, products, places) and the links between them (“works at”, “uses”, “depends on”). Once the facts are in a graph, you can ask “Who works at Acme?” or “Which products depend on PostgreSQL?” and get the answer in a second, across a thousand files.

Pulling the things and the links out of the text is called **extraction**. It is the hard part.

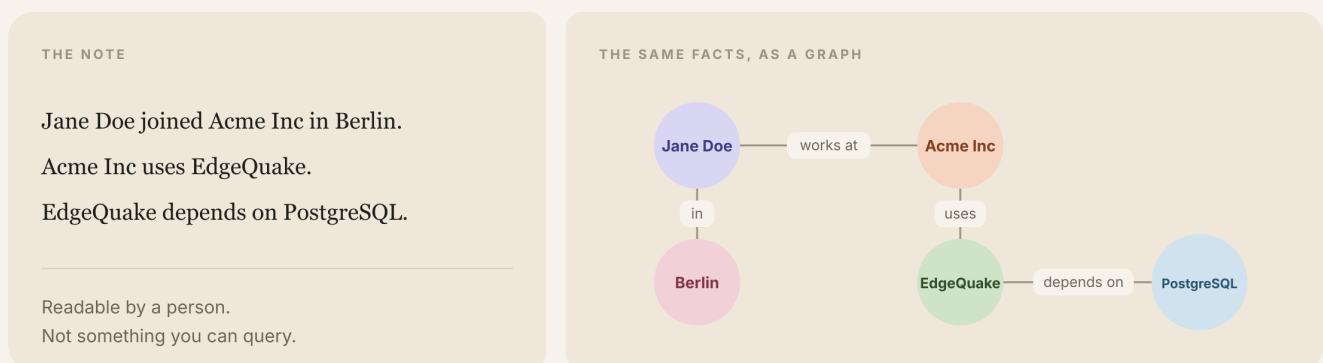


FIGURE 1 A note, and the same facts as a graph.

02

Why it is hard to do well today

There are five common ways to do it. Each one gives something up.

Hand-written rules	Regex and word lists. Cheap and fast. Breaks on new names and new phrasing.
Classic NLP taggers	Trained on news or biomedical text. Breaks when your list of kinds is different.
Trained models	Best scores on the benchmark they trained on (SpERT-class). You must label data, then retrain for each domain.
Chat model, JSON	Fast to try, and works on anything. Gives no probability. Keeps every guess it writes.
This method	Code proposes. Your ontology limits. A decision model says yes or no. Your cutoff keeps, reviews, or drops each link. THE METHOD IN THIS ARTICLE

FIGURE 2 Five ways to find names and links in text. The last row is the method in this article.

Rules (word lists and search patterns) are cheap, but they break the moment someone phrases a sentence differently. **Older trained tools** work well on the kind of text they were trained on, usually news, and badly on anything else. **Newer trained models** give the best scores, but only after someone labels thousands of examples for your topic.

The fourth way is the popular one: ask a **chat model** to write the answer as JSON, a tidy format that programs can read. It works on almost anything and takes ten minutes to try. The catch is easy to miss. A tidy answer is not a true answer. The model is still just writing the next word. It does not tell you how sure it is, so a guess looks exactly like a fact, and you end up reading every link yourself.

03

The idea: ask small questions, get a number

A **decision model** is a small AI model that does not write. It answers one **closed question**. "Closed" means you decide what shape the answer can take before the model runs. There are three shapes.

Pick one (called Choice)	Choose one label from a short list you wrote. returns: the winner, and how sure it is of every label
Yes or no (called Noul)	Answer a yes-or-no question. Does EdgeQuake use PostgreSQL? -> 0.99
Give a level (called Score)	Rate something on a short scale you wrote. returns: the expected level, and how sure it is of each

FIGURE 3 Three kinds of closed question. The technical names are in small print.

The answer always comes with a number between 0 and 1 that says how sure the model is.

Here is a real call, with real output, from the model `tev1` running on one computer:

```
Question 1: Does EdgeQuake use PostgreSQL? -> 0.99 (very sure: yes)
Question 2: Did Acme Inc found PostgreSQL? -> 0.01 (very sure: no)

Time: 2.9 seconds. Words written by the model: 3 tokens.
```

A **token** is a small piece of text, about three-quarters of a word. For comparison, Mistral Small, a chat model, wrote about 250 tokens for each news sentence we gave it, because it writes out every link in full.

Two things to know before you trust the number. First, it is not an exact odds. A 0.9 does not mean “right nine times in ten”. You have to check it against your own examples. Second, it needs the right tools.

Ollama is a free program that runs AI models on your own computer. Since version 0.35 it has a special door for this kind of model, called `/v1/systemone`. We measured the models `tev1` and `nimble`. We have not tested `clef` or `clef-flash`.

A DECISION ENDPOINT

POST `/v1/systemone`

You declare the answer type first.
You get a value inside it, with a probability.
Your code still decides what to keep.

A JSON COAT ON A CHAT MODEL

`format / schema / grammar`

It forces a legal string.
A legal string can still be the wrong branch.
The model is still writing the next token.

FIGURE 4 A decision model and a chat model with a JSON format are not the same thing.

Forcing a chat model to output JSON makes the answer well-formed. It does not make it true, and it does not give you a number you can use as a cutoff.

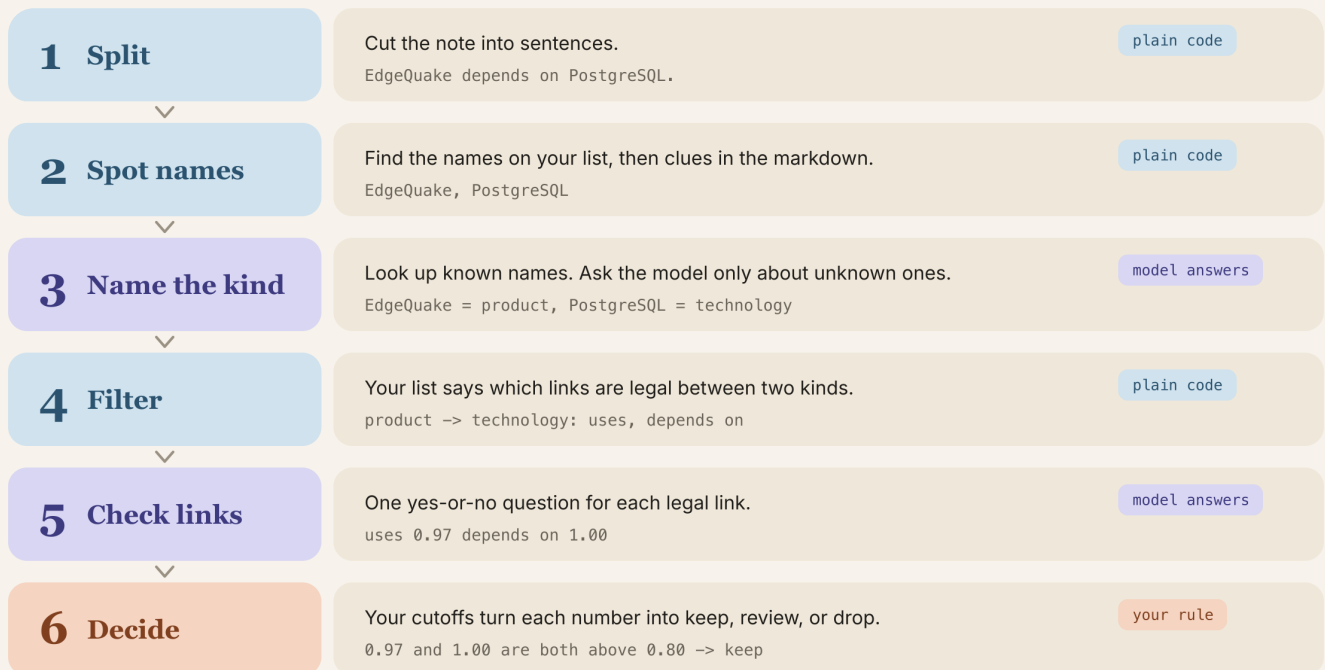
04

How edgextract works

edgextract is a small Python library. You give it a markdown file and a list. It gives you a graph. The list is called an **ontology**: you write down the kinds of thing you care about (person, company, place) and which links are allowed between them (a person can work for a company, a company cannot work for a place).

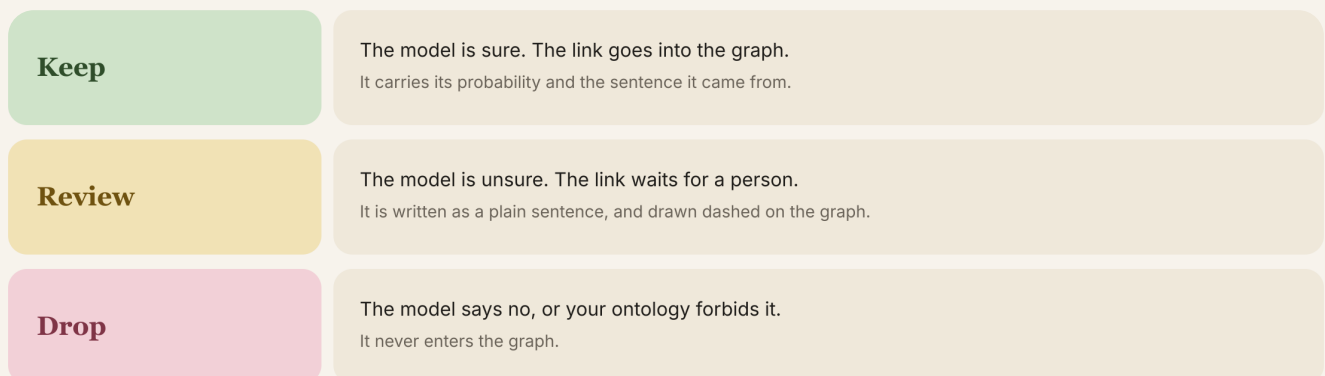
The tool works in six steps, shown in the figure below on one sentence: “EdgeQuake depends on PostgreSQL.” Three ideas hold the design together.

- **The code finds, the model only judges.** Names come from your list and from markdown clues, such as bold text. The model is asked about a name only when the code has never seen it.
- **Your list shrinks the questions.** It says a product can use or depend on a technology. It says nothing about a place depending on a person, so that pair is never asked. This is why the tool cannot invent a kind of link.
- **Each link is its own question.** “Uses” and “depends on” can both be true, so each gets its own yes-or-no. Here the answers were 0.97 and 1.00.



Worked example: the sentence "EdgeQuake depends on PostgreSQL." goes through all six steps.

FIGURE 5 Six steps, with one sentence followed all the way through. Plain code finds, the model answers closed questions, your rule makes the final call.

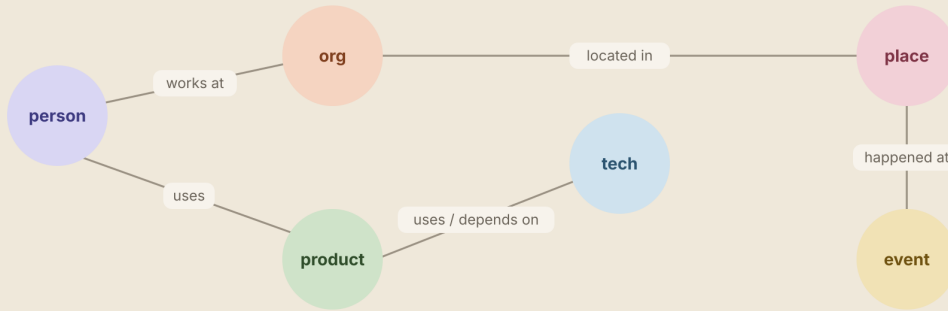


Your code owns the two cutoffs. They start unfitted: fit them on your own labeled notes.

FIGURE 6 A link is kept when the model is sure, sent to a person when it is not, and dropped when the answer is no.

In the default setting a yes at 0.80 or higher is kept, a no at 0.20 or lower is dropped, and everything in between waits for a person as a plain sentence. These cutoffs are your code's, not the model's. They start as sensible guesses and are not fitted to your data. Fit them on your own examples before you rely on them.

THE LIST YOU WRITE: KINDS OF NAME, AND THE LINKS BETWEEN THEM



A link outside this list is never asked, so it can never appear.

FIGURE 7 A few of the links the software-documentation list allows. The full list has seven kinds of link.

The model never finds a name by itself and never invents a kind. It only answers the questions the code asks.

05

Write your own list in ten minutes

The same software works on any topic. A second list ships with it for company news: people, companies, and places, with links such as founded, acquired, invested in, and works for.

```
edgextract init-ontology my_list.yaml      # writes a starter file with comments
edgextract validate-ontology my_list.yaml  # prints your kinds and links in plain words
edgextract --model tev1 report note.md --ontology my_list.yaml --out graph.html
```

Or from Python:

```
from edgextract import extract_text, load_ontology_named, write_report

text = open("note.md").read()
ontology = load_ontology_named("company_news")
result = extract_text(text, ontology, model="tev1")
write_report("graph.html", text, result, ontology, title="My note")
```

06

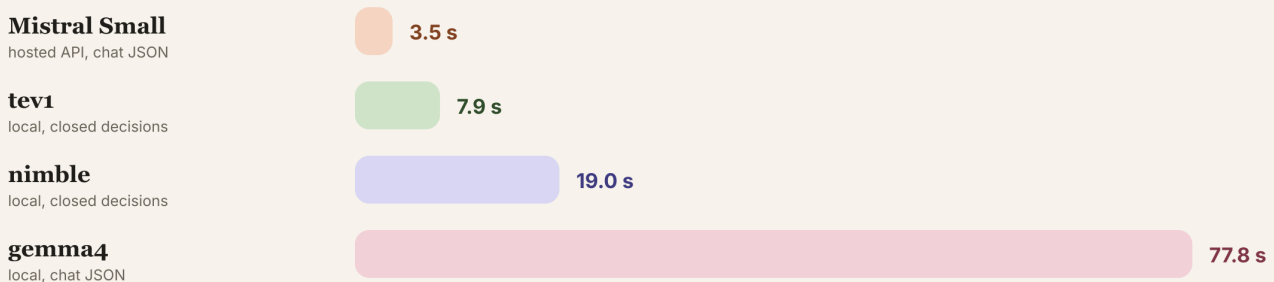
What we measured

Every number here was produced on one computer on 3 October 2026. The commands are in the repository. Most results below use a **score** between 0 and 1. It goes up when the tool finds the right items, and down when it keeps wrong ones. 1.00 is a perfect match with the answer key. We also give raw counts, because they are easier to understand than a score.

One short software note

We took a seven-sentence note about EdgeQuake and compared four ways of doing the job. The names on this note are on the list, so a perfect name score here only shows that lookup works.

METHOD	NAMES	LINKS	SECONDS
tev1 , one yes-or-no per link	1.00	0.78	7.9
nimble , the same questions	1.00	0.38	19.0
Mistral Small, JSON (hosted service)	0.89	0.80	3.5
gemma4 , JSON (same computer)	0.95	0.89	77.8



One seven-sentence software note, one machine. A hosted model is fast but sends your text out and bills by the token.

FIGURE 8 Seconds to process one note. The hosted model is the fastest. Among models on your own computer, the decision models are far faster.

Read this fairly. On a single note, a chat model matched or beat the decision model on links. The decision model's advantage here is speed against a chat model on the same computer, plus the numbers and the review list. A hosted service is fast too, but it sends your text out and charges for every token.

Twelve software notes

We also ran both methods on the 12 hand-labeled software notes that ship with the repository. They include awkward cases on purpose: a note with no facts, a negation ("X does not use Y"), a code fence, a hedge, pronouns, and text that tries to give the model orders.

METHOD	NAMES SCORE	LINKS SCORE	RIGHT LINKS	WRONG LINKS	SECONDS, ALL 12
edgextract with tev1	0.97	0.59	11 of 18	8	12.7
Mistral Small, JSON	0.68	0.43	10 of 18	19	22.1

On two company-news notes both methods found all 10 names and 6 of 6 links, each with one extra link.

Treat this as a sanity check, not a fair contest. We used these notes while building the tool, and the tool finds names by lookup and markdown clues, which helps it here. Chat-model answers also change from run to run: an earlier run of Mistral Small scored 0.80 on names and 0.38 on links on the same notes. The `tev1` figure for seconds is with the model already loaded; the first call is slower.

A public news test

CoNLL04 is a well-known set of news sentences that researchers use to compare tools. People marked every name and link by hand. We used the 288 sentences set aside for testing. None of the names were given to the tool in advance, so it had to find them. A name counts only if the exact words and the kind are right. A link counts only if both names and the type are right. The answer key has 1,079 names and 422 links.

METHOD	NAMES SCORE	LINKS SCORE	RIGHT LINKS FOUND	WRONG LINKS KEPT
A model trained on this test's training sentences (SpERT)	0.89	0.71	not measured here	not measured here
edgextract with <code>tev1</code> , final run	0.69	0.39	154	218
edgextract with <code>tev1</code> , first run	0.66	0.41	155	177
Mistral Small, JSON	0.64	0.33	183	504

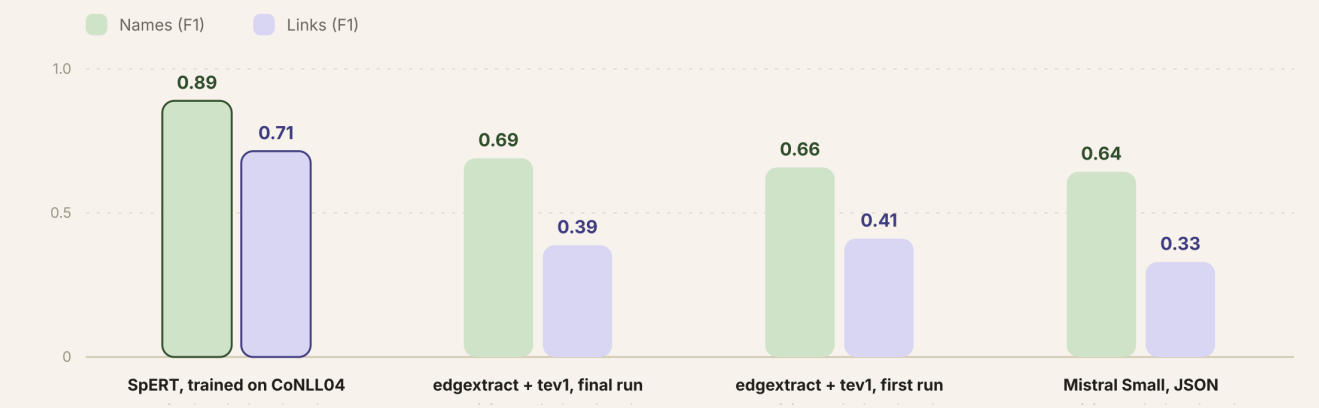


FIGURE 9 Scores on the news test. The trained model has seen this kind of text before; the others have not.

Here is the fair reading.

- **A trained model still wins.** If you have labeled examples in your own domain, train a model. That is the right tool for the job and we say so.
- **Against a chat model, the decision model keeps far fewer wrong links.** In the final run it kept 218 wrong links where Mistral Small kept 504, while finding 154 right ones against 183. It gives up a little recall to avoid a lot of noise.

- **We ran it twice and report both.** The first run used a small name-finding model. For the second we switched to a larger one and changed to one question per link. The larger finder helped with names (correct names went from 634 to 683). It did not help with links: the score slipped from 0.41 to 0.39, and reversed links went from 9 to 16. On the development sentences the change had looked better, so the gain did not carry over to the test. We kept the final run as the number of record and did not pick the better-looking one.
- **The test was opened twice, once per configuration.** The development sentences were used for all tuning.
- **gemma4 has no news-test score.** We started it through Ollama, stopped after about ten sentences when we moved the comparison to Mistral Small, and do not count that partial run.

What it costs to run

	DECISION MODEL ON YOUR COMPUTER	CHAT MODEL ON A HOSTED SERVICE
Money	Your own hardware, no per-token bill	Charged per token
Where your text goes	Stays on your computer	Sent to the provider
Speed on the news test	Seconds per sentence	1.8 seconds per sentence
Wrong links kept (news test)	218	504
What it does with an unsure answer	Puts it in a review list	Keeps it
New topic	Edit the list	Rewrite the prompt and hope

07

Where it gets things wrong

The tool makes mistakes, and the number does not always warn you.

- **Direction.** On the EdgeQuake note it kept both "Ollama uses Nimble" (0.97, which is right) and the reversed "Nimble uses Ollama" (0.91, which is wrong). A confident number can be confidently wrong.
- **Look-alike links.** In the Northwind sample it kept "Acme Inc acquired Northwind", but the text says Acme invested in it. That one is wrong, and it is still in the sample graph in this repository on purpose.
- **Extra links.** It also kept "Jane Doe is part of Acme Inc" at 0.96, which the answer key does not contain.
- **One sentence at a time.** Links across sentences are not joined, and pronouns such as "she" or "it" are skipped.
- **It needs Ollama 0.35 or newer** and a decision model on your computer.

The review list helps, but it cannot catch a mistake the model is sure about. Treat the graph as a strong first draft, not a finished fact base.

See the graph

Every run can write one self-contained web page, with no internet needed. It shows the text with the names highlighted, the links written as plain sentences, the items waiting for a person, and a graph you can move. Drag a name, scroll to zoom, click a name to read its links, hover a line to read the sentence behind it. Dashed amber lines are waiting for a person.

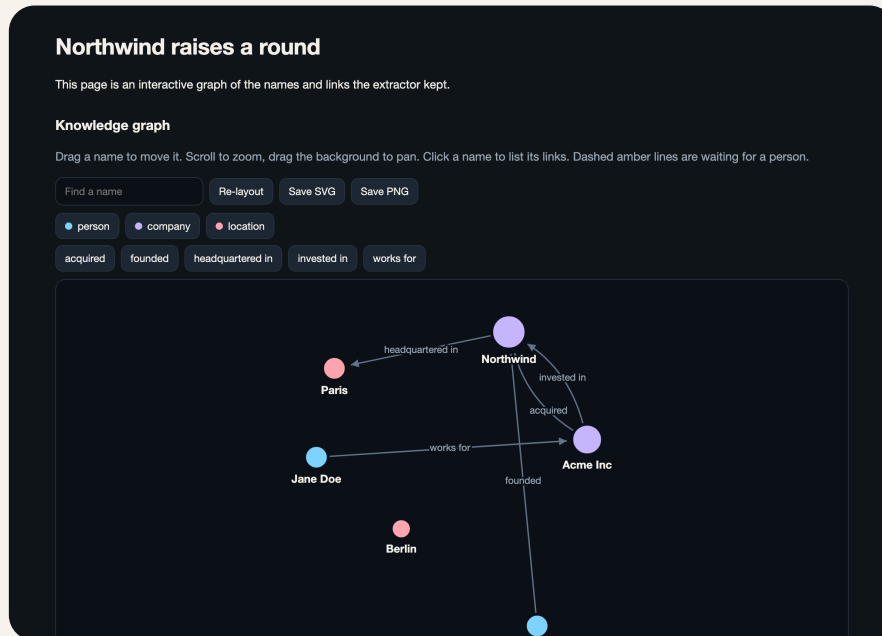


FIGURE 10 The interactive graph from a real run on a short company-news note. One of the five links (Acme acquired Northwind) is wrong.



FIGURE 11 Unsure items wait as plain sentences. A person keeps or drops each one.

```
edgextract --model tev1 report note.md --ontology company_news --out graph.html
```

Which one should you use

Use edgextract	You can write down your kinds and links, and you cannot label thousands of sentences. You want an unsure answer to wait for a person instead of entering the graph.
Train a model	You have labeled sentences in your domain and you need the best score. On CoNLL04 a trained model scores 0.89 and 0.71. Nothing here does.
Ask a chat model	You are exploring, your list of kinds keeps changing, or a wrong link is cheap to delete. Expect extra links. Check them.

FIGURE 12 Pick the tool that fits what you have. All three are reasonable.

Try it

```
git clone https://github.com/raphaelmansuy/edgextract && cd edgextract
uv sync --all-extras
ollama pull tev1
edgextract probe --model tev1
edgextract --model tev1 report data/golden/docs/13_northwind.md --ontology company_news --out
graph.html
make test examples          # no model needed: uses a stand-in
```

Nine short examples teach one idea each. The full evaluation, with every number and the command that produced it, is in `specs/0001-implementation/14-evaluation.md`.

The cutoff is yours.

Refit it when the model, the wording, or your list changes.

Written by **Raphael MANSUY** · [linkedin.com/in/raphaelmansuy](https://www.linkedin.com/in/raphaelmansuy)